.NET 工程师面试 Q&A

基于简历内容整理，包含项目介绍、技术细节、常见面试题

一、项目介绍类

1. 请简单介绍一下你最近做的项目

参考回答： 我最近在河南产权交易中心担任后端开发工程师，负责多个独立系统的需求分析、架构设计与落地。主要包括：

- 专家培训与考试系统（.NET 10 + Redis + WebSocket，支持高并发在线考试）
- 多租户电子发票网关系统（支持多渠道开票，MongoDB 缓存）
- 金融保函核心业务系统（工厂+策略模式多资方接入）

在此之前，我在杭州智瑞思负责 HIS 医院信息系统的中药房/药房模块开发，使用 WPF + MVVM 架构，实现了药品入库、出库、发药、设备对接等功能。

---

2. 你在 HIS 项目中负责什么模块？具体做什么？

参考回答： 在杭州智瑞思期间，我负责 HIS 系统中药房/药房模块的桌面端开发。

工作内容：

- 药品基础信息管理（中药、西药）
- 采购入库、出库、退药、盘点
- 处方发药（从医生站接收处方 -> 配药 -> 发药确认）
- 药品调价、效期管理
- 与自动发药机、煎药机、电子秤等设备进行串口通信（RS232/RS485）
- 数据同步（与挂号、收费、住院护士站、医生站等模块交互）

技术实现：

- 使用 WPF + MVVM（CommunityToolkit.Mvvm）重构原 WinForms 模块
- 针对医院内网环境优化性能（本地缓存、索引优化、异步加载）

---

3. 专家考试系统解决了哪些技术难点？

参考回答： 主要解决了以下几个技术难点：

1. **高并发登录** - 使用 Redis 滑动窗口实现短信登录防刷限流，峰值期登录稳定率 99%+

2. **大文件上传** - 七牛云分片直传 + 断点续传，弱网环境上传成功率提升约 30%

3. **异步转码** - Redis 队列 + Webhook 构建转码管道，转码任务平均等待时长下降约 40%

4. **实时消息推送** - 多节点 WebSocket 通过 Redis Pub/Sub 跨节点路由，保障实时推送一致性

5. **审批流引擎** - 基于 Workflow Core 搭建通用审批流，支持按部门/角色配置权限与层级自动流转

二、技术细节类

4. WPF MVVM 项目中，如何实现 ViewModel 之间的通信？

参考回答： 常用几种方式：

1. **Messenger（消息机制）** - 使用 CommunityToolkit.Mvvm 的 WeakReferenceMessenger

```
// 发送方
WeakReferenceMessenger.Default.Send(new RefreshDataMessage());

// 接收方
WeakReferenceMessenger.Default.Register<RefreshDataMessage>(this, (r, m) => {
    LoadData();
});
```

2. **依赖注入** - 通过构造函数注入其他 ViewModel 或服务
3. **事件聚合器** - Mediator 或 Prism 的 EventAggregator
4. **共享服务** - 将需要通信的数据放在单例服务中

项目中实践： HIS 模块中，处方发药完成后，通过 Messenger 通知库存管理界面刷新库存数据。

---

5. 串口通信（RS232）在HIS中如何应用？

参考回答： 在医院场景中，串口常用于对接药房设备：

1. **自动发药机** - 处方配药完成后，发送指令让发药机弹出对应药盒
2. **煎药机** - 接收煎药处方，监控煎药进度与完成状态
3. **电子秤** - 药品称重（中药饮片称量）

实现方式：

```
// 串口配置（常用参数）
SerialPort serialPort = new SerialPort("COM1", 9600, Parity.None, 8, StopBits.One);
serialPort.DataReceived += SerialPort_DataReceived;

// 发送指令
serialPort.Write(command, 0, command.Length);

// 接收数据
void SerialPort_DataReceived(object sender, SerialDataReceivedEventArgs e)
{
    byte[] buffer = new byte[serialPort.BytesToRead];
    serialPort.Read(buffer, 0, buffer.Length);
    // 解析协议
}
```

注意事项：

- 医院设备多为 RS232 或 RS485，需考虑电平转换
- 设备协议通常是自定义二进制协议，需按协议文档解析
- 串口是独占资源，需做好异常处理与断开重连
- 建议使用单独的线程处理串口通信，避免阻塞 UI

---

6. Redis 在项目中如何使用的？

参考回答： 主要用在以下几个方面：

1. **缓存** - 缓存字典数据、热点数据（如科室、药品基础信息）
2. **限流** - 短信登录防刷（滑动窗口）

```
// 滑动窗口限流
var key = $"login:rate:{phone}";
var now = DateTimeOffset.UtcNow.ToUnixTimeSeconds();
await redis.zAddAsync(key, (now, now.ToString()));
await redis.zRemRangeByScoreAsync(key, 0, now - 60); // 保留60秒窗口
var count = await redis.zCardAsync(key);
```

3. **消息队列** - 异步转码任务队列
4. **Pub/Sub** - WebSocket 多节点消息路由
5. **分布式锁** - 避免并发冲突（如库存扣减）

---

7. 你如何做性能优化？

参考回答： 从几个维度：

1. **数据库层面**
   - 索引优化（explain 分析慢查询）
   - 分页查询避免全表扫描
   - 读写分离（针对报表查询）
2. **缓存层面**
   - 热点数据 Redis 缓存
   - 本地内存缓存（字典数据）
   - 缓存失效策略（TTL）
3. **代码层面**
   - 异步编程（async/await）
   - 减少不必要的对象创建
   - 批量操作（BulkInsert）
4. **项目中实践**
   - HDIS 项目：复杂报表查询索引优化 + Redis 缓存，响应从约 1000ms 优化至约 200ms
   - 专家考试系统：分片上传、转码异步队列

三、数据库与设计模式

8. 你用过哪些设计模式？项目中如何应用？

参考回答：

1. **工厂模式** - 电子发票网关：标准化第三方开票渠道，实现服务商「即插即用」
2. **策略模式** - 金融保函：抽象多资方接入逻辑，不同资方使用不同策略
3. **装饰器模式** - 日志记录增强
4. **单例模式** - 配置管理、缓存服务

5. **观察者模式** - 事件通知（MVVM Messenger 底层）

---

9. EF Core 和 Dapper 的区别？项目中如何选择？

参考回答：

特性	EF Core	Dapper
性能	中等（对象映射开销）	极高（轻量）
开发效率	高（ORM、全功能）	低（需手写 SQL）
复杂查询	一般	灵活
迁移支持	有	无

项目中应用：

- HDIS 项目：使用 SqlSugar（类似 EF Core 的国产 ORM）
- 高性能场景：Dapper 或直接 ADO.NET
- 一般 CRUD：EF Core / SqlSugar

四、场景题

10. 高并发情况下如何保证库存不超卖？

参考回答： 几种方案：

1. **数据库乐观锁**

```
// UPDATE Inventory SET Stock = Stock - 1 WHERE Id = @id AND Stock > 0
```

2. **Redis 分布式锁**

```
var lock = await redis.acquireLockAsync("inventory:lock:" + productId);
if (lock != null) {
    // 扣减库存
    await redis.decrAsync("inventory:" + productId);
    await lock.ReleaseAsync();
}
```

3. **Redis 原子操作**

```
var result = await redis.ExecuteAsync("DECR", "inventory:" + productId);
if (result < 0) {
    // 库存不足，回滚
    await redis.incrAsync("inventory:" + productId);
}
```

实际项目中做法：

- 先扣 Redis 库存，异步同步到数据库
- 数据库采用乐观锁，失败则回滚 Redis 并重试

---

11. 如何设计一个通用审批流？

参考回答：

基于 Workflow Core 的设计思路：

1. **流程定义**
    - 节点：审批节点（按人/角色/部门）、分支节点、结束节点
    - 连线：节点之间的流转条件
2. **流程实例**
    - 存储当前节点、审批人、审批状态
3. **核心表结构**
    - WorkflowDefinition（流程定义）
    - WorkflowInstance（流程实例）
    - WorkflowTask（待办任务）
    - WorkflowHistory（审批历史）
4. **实现要点**
    - 按部门/角色/身份动态分配审批人
    - 支持会签（多人审批）、或签（任一人审批）
    - 审批完成后自动流转到下一节点

---

12. 如果系统出现 OOM，如何排查？

参考回答：

1. **首先确认 OOM 类型**
    - 托管堆 OOM（.NET 对象未释放）
    - 非托管内存 OOM（Native 泄漏，如 GDI+、COM）
2. **排查步骤**
    - 查看 Windows 事件查看器错误日志
    - 使用 dotMemory / ANTS Memory Profiler 分析堆
    - 检查大对象是否及时释放（using、Dispose）
    - 检查是否有内存泄漏（WeakReference 弱引用）
3. **常见原因**
    - 大数据量一次性加载（如全表查询导出 Excel）
    - 静态集合（如 Dictionary）无限增长
    - 事件未取消订阅
    - 图片/大文件未及时释放
4. **HIS 项目中的实践**
    - 大数据量查询采用分页或流式处理
    - 使用 using 释放 SerialPort、Database 连接

五、反问与反套路

13. 你有什么问题想问我的？

参考建议：

- 技术栈与技术选型
- 团队规模与分工
- 项目业务复杂度
- 技术成长空间

示例：

- 请问贵司的后端技术栈主要是什么？
- 这个岗位主要负责哪个业务模块？
- 团队目前遇到比较大的技术挑战是什么？

六、HIS 业务知识速查

常见模块与业务流程

模块	主要功能
挂号	门诊/住院登记、卡管理、分诊
收费	划价、收费、退费、结算
医生站	门诊/住院病历、处方开立、检查检验申请
护士站	医嘱执行、护理记录、床位管理
药房	入库、出库、发药、配药、盘点
药库	采购、库存管理、效期预警
检验/检查	标本采集、结果录入、报告打印
手术室	手术安排、麻醉记录
物资/设备	库存管理、采购、折旧

核心数据流

```
患者挂号 --> 医生站就诊 --> 开具处方/检查检验 --> 收费处缴费
--> 药房取药 / 检验科检查 --> 完成就诊
```

建议：根据实际项目经历，选择性参考以上答案，重点突出解决问题的能力和技术亮点。